

$$C_x(k) = \frac{1}{N} \sum_{m=0}^{N-1} X(m) W^{km}, \text{ где } k = \overline{0, N-1}, W = e^{-i2\pi/N}, i = \sqrt{-1}$$

$$X(m) = \sum_{k=0}^{N-1} C_x(k) W^{-km}.$$

$x(m)$ – входной сигнал во временной области

$C_x(k)$ – сигнал в частотной области (спектральные коэффициенты)

N – длина входной последовательности

$W = e^{-j2\pi km/N}$

k – индекс частоты

m – временная точка

Можно выделить следующие области применения ДПФ:

- цифровой спектральный анализ
 - о анализаторы спектра
 - о обработка речи
 - о обработка изображений
 - о распознавание образов
- проектирование фильтров
 - о вычисление импульсной характеристики по частотной
 - о вычисление частотной характеристики по импульсной
- быстрое преобразование Фурье (БПФ) – простой алгоритм для эффективного вычисления ДПФ.

Ортогональность – это свойство двух функций, при котором их скалярное произведение равно нулю.

DFT строится на базе набора ортогональных функций – комплексных экспонент $e^{-j2\pi km/N}$. Каждая из этих экспонент представляет синусоиду определённой частоты. Благодаря ортогональности каждая частотная компонента вычисляется независимо от других.

- **DFT** разлагает сигнал на частоты.
- **IDFT** восстанавливает сигнал обратно.
- **Коэффициент W** – который позволяет связать временную и частотную области
- **Нормализация** важна, чтобы сигнал после преобразований не изменял амплитуду.

```
import numpy as np # Импортируем библиотеку numpy для работы с
массивами и математическими операциями
```

```
import matplotlib.pyplot as plt # Импортируем matplotlib для
построения графиков
```

```
from scipy.io.wavfile import write # Импортируем функцию write из
scipy.io.wavfile для сохранения аудиофайлов
```

```
# Реализация дискретного преобразования Фурье (DFT)
```

```
def dft(x):
```

```
    N = len(x) # Определяем длину входного сигнала (количество точек)
```

```
    W = np.exp(-2j * np.pi / N) # Вычисляем комплексный множитель W
(основной поворотный коэффициент)
```

```
    Cx = np.zeros(N, dtype=complex) # Создаём массив комплексных
чисел для хранения коэффициентов DFT
```

```
    for k in range(N): # Проходим по всем частотным компонентам
```

```
        for m in range(N): # Проходим по всем временным точкам
входного сигнала
```

```
            Cx[k] += x[m] * W ** (k * m) # Выполняем суммирование с
умножением на экспоненциальный множитель
```

```
            Cx[k] /= N # Нормализуем результат, деля на N (чтобы
амплитуда сигнала не увеличивалась)
```

```
    return Cx # Возвращаем спектральные коэффициенты
```

```
# Реализация обратного дискретного преобразования Фурье (IDFT)
```

```
def idft(Cx):
```

```
    N = len(Cx) # Определяем длину спектрального сигнала
```

```
    W = np.exp(-2j * np.pi / N) # Вычисляем комплексный множитель W
```

```

    x = np.zeros(N, dtype=complex) # Создаём массив комплексных чисел
    для восстановления сигнала во временной области

    for m in range(N): # Проходим по всем временным точкам
        for k in range(N): # Проходим по всем частотным компонентам
            x[m] += Cx[k] * W ** (-k * m) # Выполняем суммирование с
            обратным экспоненциальным множителем

    return x # Возвращаем восстановленный сигнал

# Определяем параметры сигнала
duration = 1.0 # Длительность сигнала в секундах
sampling_rate = 500 # Частота дискретизации (сколько отсчётов в
секунду)
t = np.linspace(0, 2 * np.pi * duration, int(sampling_rate *
duration))

# Создаём массив временных точек от 0 до  $2\pi * duration$  с заданным
количеством точек

# Определяем сам сигнал: сумма синуса и косинуса
signal = np.sin(t) + np.cos(4 * t) # Формируем сигнал как сумму
синусоидальных составляющих

# Визуализируем исходный сигнал
plt.figure(figsize=(10, 4)) # Создаём область для графика с заданным
размером
plt.plot(t, signal) # Строим график сигнала
plt.title('Исходный сигнал:  $y = \sin(x) + \cos(4x)$ ') # Заголовок
графика
plt.xlabel('Время [с]') # Подпись оси X (время)
plt.ylabel('Амплитуда') # Подпись оси Y (амплитуда)
plt.grid(True) # Включаем сетку на графике
plt.show() # Отображаем график

```

```

# Масштабируем сигнал для сохранения в аудиофайл (16-битное
представление)

scaled_signal = np.int16(signal / np.max(np.abs(signal)) * 32767)

# Нормализуем сигнал на диапазон [-32767, 32767] (16-битный звук)

write("output_signal.wav", sampling_rate, scaled_signal) # Сохраняем
сигнал в WAV-файл

# Выполняем дискретное преобразование Фурье (DFT)
dft_signal = dft(signal)

# Вычисляем массив частот, соответствующих компонентам DFT
frequencies = np.fft.fftfreq(len(signal), d=(t[1]-t[0]))

# np.fft.fftfreq вычисляет соответствующие частоты для коэффициентов
преобразования Фурье

# Вычисляем амплитуды спектральных коэффициентов
magnitude = np.abs(dft_signal) # Модуль комплексных коэффициентов DFT
даёт амплитудный спектр

# Выделяем только положительные частоты (симметричная половина
спектра)
positive_frequencies = frequencies[:len(frequencies)//2] # Берём
только первую половину массива частот
positive_magnitude = magnitude[:len(magnitude)//2] # Аналогично берём
первую половину спектра амплитуд

# Визуализируем амплитудно-частотную характеристику (АЧХ)
plt.figure(figsize=(10, 4)) # Создаём область для графика
plt.stem(positive_frequencies, positive_magnitude) # Строим график
частотного спектра (столбчатая диаграмма)
plt.title('Амплитудно-Частотная Характеристика (АЧХ)') # Заголовок
графика
plt.xlabel('Частота [Гц]') # Подпись оси X (частота)

```

```
plt.ylabel('Амплитуда') # Подпись оси Y (амплитуда)

# Ограничиваем диапазон частот и амплитуд для наглядности
plt.xlim(0, 1) # Ограничиваем ось X до 1 Гц
plt.ylim(0, 1) # Ограничиваем ось Y до 1
plt.xticks(np.arange(0, 1, 0.1)) # Устанавливаем шаг меток по оси X

plt.grid(True) # Включаем сетку
plt.show() # Отображаем график

# Ослабляем спектральные коэффициенты в 2 раза
dft_signal_reduced = dft_signal / 2 # Делим все коэффициенты DFT на
2, ослабляя амплитуды частотных компонент

# Выполняем обратное дискретное преобразование Фурье (IDFT)
reconstructed_signal = idft(dft_signal_reduced)

# Восстанавливаем сигнал обратно во временную область (но с
ослабленными частотными компонентами)

# Визуализируем восстановленный сигнал
plt.figure(figsize=(10, 4)) # Создаём область для графика
plt.plot(t, np.real(reconstructed_signal)) # Строим график
восстановленного сигнала (берём только реальную часть)
plt.title('Восстановленный сигнал после обратного DFT') # Заголовок
графика
plt.xlabel('Время [с]') # Подпись оси X (время)
plt.ylabel('Амплитуда') # Подпись оси Y (амплитуда)
plt.grid(True) # Включаем сетку
plt.show() # Отображаем график

# Масштабируем восстановленный сигнал для сохранения в аудиофайл
scaled_reconstructed_signal = np.int16(np.real(reconstructed_signal) /
np.max(np.abs(reconstructed_signal)) * 32767)
```

```
# Приводим восстановленный сигнал к 16-битному диапазону [-32767, 32767]
```

```
write("reconstructed_signal.wav", sampling_rate, scaled_reconstructed_signal)
```

```
# Сохраняем восстановленный сигнал в WAV-файл
```